Blossom Full-Stack Technical Assessment

1. Execution Instructions (README)

Docker (Full-Stack Environment)

To run the database, backend, and frontend simultaneously using Docker Compose:

docker-compose up -d --build

Backend (Spring Boot) - Manual Execution

- Requirements: Java 21+, PostgreSQL (or Docker for DB).
- Start DB: docker-compose up -d postgres-db
- Run App: ./gradlew bootRun
- Swagger API Docs: http://localhost:8080/swagger-ui/index.html

Frontend (React + Vite) - Manual Execution

- Requirements: Node.js 24+
- Install dependencies: npm install
- Run dev server: npm run dev
- App URL: http://localhost:5173

2. Architecture Layers

The backend follows a strict Layered Architecture (N-Tier) to ensure separation of concerns, testability, and scalability:

1. Presentation Layer (Controllers): Handles incoming HTTP requests, validates DTOs, and returns HTTP responses. Exposes the REST API for the React frontend.

2. Security Layer (JWT & Filter): Intercepts requests, validates JWT tokens, and manages authentication/authorization context (e.g., JwtAuthenticationFilter, SecurityConfig).

3. Business Logic Layer (Services): Contains the core logic, transaction management (@Transactional), and DTO-to-Entity mapping. Enforces business rules.

4. Data Access Layer (Repositories): Interfaces extending JpaRepository. Handles database interactions, custom JPQL queries, and N+1 prevention (@EntityGraph).

5. Data Layer (Entities): JPA models mapped to PostgreSQL tables (User, Product, Order, OrderItem).

3. Database Entity-Relationship (ER) Diagram

The following represents the relational data model for the e-commerce platform. It implements Single Table Inheritance for Products.

```
USERS ||--o{ ORDERS : places
  USERS {
    Long id PK
    String email UK
    String password
  }

  ORDERS ||--o{ ORDER_ITEMS : contains
  ORDERS {
    Long id PK
    DateTime orderDate
    String status
    Long user_id FK
  }

  ORDER_ITEMS }o--|| PRODUCTS : includes
  ORDER_ITEMS {
    Long id PK
    Integer quantity
    Decimal unitPrice
    Long order_id FK
    Long product_id FK
  }

  PRODUCTS {
    Long id PK
    String product_type "Discriminator"
    String name
    String category
    Decimal price
    Double weightKg "If PHYSICAL"
    String shippingDimensions "If PHYSICAL"
    String downloadLink "If DIGITAL"
    Double fileSizeMb "If DIGITAL"
  }
```

4. Design Decisions & Challenges

- Data Architecture: Implemented JPA inheritance (SINGLE_TABLE) to separate PhysicalProduct and DigitalProduct, keeping a unified catalog while allowing specific attributes.
- Security: Built a custom JWT authentication filter for stateless sessions, securing private endpoints seamlessly.

- N+1 Query Prevention: Solved potential performance bottlenecks in the Order History endpoint by using @EntityGraph to perform a JOIN FETCH on order items.
- UI State Management: Chosen TanStack Query (React Query) over traditional Redux for server-state, caching, and async handling. Native Context API was used for local synchronous state (Shopping Cart).

5. Theoretical Questions

1. How would you implement the Repository pattern in this project?

It was implemented by creating interfaces that extend Spring Data's JpaRepository (e.g., ProductRepository). This abstracts all persistence logic and SQL queries. Any database interaction is done through these methods, allowing the service layer to focus solely on business logic, adhering to the Single Responsibility Principle.

2. Differences between JPA and Hibernate and when to use each.

JPA (Java Persistence API) is the Java specification/standard for Object-Relational Mapping (the "what").

Hibernate is the concrete implementation of that specification (the "how").

In a modern project, I would always use JPA (Spring Data JPA) as the public interface in my code to remain framework-agnostic, allowing Spring to use Hibernate under the hood transparently. I avoid coupling code directly to Hibernate-specific classes unless very low-level optimizations are required.

3. What advantages does React offer compared to other frontend frameworks?

- Virtual DOM: Optimizes rendering by updating only changed nodes, boosting performance in dynamic lists like our product catalog.
- Extensive Ecosystem: Allows using top-tier, purpose-built libraries (like TanStack Query for data fetching or react-hook-form for validation) rather than monolithic solutions.
- Reusable Components: Encourages atomic design, making scalable UI creation easier.
- Unidirectional Data Flow: Makes data behavior predictable (parent to child), which simplifies debugging.

4. Explain how you would organize your time over 2 days to deliver this assignment.

- Day 1 (Backend & Modeling): (0-4h) Relational database design, initial configuration, and public endpoints (Login/Register with JWT). (4-8h) Product CRUD development, inheritance handling, Order controller with @EntityGraph, global exception handling (@ControllerAdvice), and Swagger configuration.
- Day 2 (Frontend & Quality): (0-4h) Vite + Tailwind setup. Base layout creation, useProducts with TanStack Query, form validation, and Shopping Cart with Context

API. (4-6h) Unit tests (JUnit, MockMvc, Vitest). (6-8h) Writing README, theoretical answers, E2E checks, Docker environment setup, and demo recording.